

=====

THE HISTORY OF

=====

ERLANG

Telephony, lightweight processes, and letting it crash: the language Ericsson built for systems that must not stop

a whitepaper in the TheHistoryOf series · 2026-07-27
source: src/02-erlang.toml (also embedded in
every page's tEXt chunk; final page is a scannable
PrintablePrograms page-frame carrying this source)

THE PROBLEM: TELEPHONE SWITCHES

=====

Erlang begins not with a theory of computation but with a machine that must never be off: the telephone exchange. In the mid-1980s the Computer Science Laboratory at Ericsson, the Swedish telecommunications company, was asked what programming technology the next generation of telecom products should use. The requirements were brutal and specific: enormous concurrency (tens of thousands of simultaneous calls), soft real-time response, software that can be upgraded without stopping the system, and tolerance of both software and hardware failure -- switches are expected to be down for minutes per decade.

Joe Armstrong, Robert Virding, and Mike Williams spent 1985-86 prototyping telephony control in every paradigm available -- Lisp, Prolog, parlog, Ada, CHILL -- and concluded that no existing language fit. Armstrong's first Erlang was a meta-interpreter on top of Prolog, which is why Erlang syntax still looks Prological. The full story is told in Armstrong's own history for HOPL III [1] and his 2003 PhD thesis, "Making

reliable distributed systems in the presence of software errors"
[2] -- the thesis title is the language's mission statement.

THE IDEAS: PROCESSES, MESSAGES, LET IT CRASH

Erlang's model is concurrency-first: the unit of program structure is the process -- not an operating-system process but a featherweight, isolated, garbage-collected actor of which a single machine can run millions. Processes share nothing and communicate only by asynchronous message passing. This is the same "little computers sending messages" picture Alan Kay drew for Smalltalk (see that chapter), arrived at independently from engineering constraints rather than biology; Armstrong liked to say Erlang might be the only truly object-oriented language.

On top of isolation sits Erlang's most famous doctrine: let it crash. Instead of defensively handling every error where it occurs, a process is linked to a supervisor whose job is to notice its death and restart it in a known-good state. Supervision trees, hot code loading (upgrading a running system), and distribution (processes on different machines talk exactly as if local) were packaged in the middle 1990s as OTP, the Open Telecom Platform -- the standard library of behaviors (`gen_server`, `supervisor`) that is, in practice, inseparable from the language.

ERLANG: THE MOVIE, THE BAN, AND OPEN SOURCE

In 1990 Ericsson produced a short internal promotional film, "Erlang: The Movie," in which Armstrong, Virding, Williams and colleagues demonstrate live call handling and hot code upgrade by phoning each other across a lab. Its deadpan delivery ("Hello Mike! Hello Joe!") made it a beloved artifact of programming history once it surfaced on YouTube [3].

The AXD301 ATM switch -- over a million lines of Erlang, famously quoted at nine nines of availability -- proved the approach at scale. Yet in 1998 Ericsson management banned new Erlang development in favor of industry-standard languages. The lab's response shaped everything after: Erlang/OTP was released as open source in December 1998, and much of the team left to found startups on it. Armstrong remained the language's voice and conscience until his death in 2019.

AFTERLIFE: FROM WHATSAPP TO ELIXIR

Erlang's second life came with the internet's need for exactly the properties telephone switches had: massive connection counts, uptime, distribution. ejabberd, RabbitMQ, CouchDB, Riak, and most famously WhatsApp -- which served hundreds of millions of users with a few dozen engineers on Erlang -- demonstrated that the telecom design generalized. The BEAM virtual machine became the industry's benchmark for preemptive, soft-real-time concurrency.

Jose Valim's Elixir (2012) put a Ruby-influenced syntax, macros, and modern tooling on the BEAM and brought the platform a new generation of programmers (Phoenix, LiveView, Nerves). Gleam (typed, 2016-) continues the line. The through-line from a 1986 Prolog meta-interpreter to today's message-passing services is unusually direct: the problems Erlang was built for turned out to be everyone's problems.

SOURCES & CITATIONS

- [1] Joe Armstrong, A History of Erlang, HOPL III, ACM, 2007
<https://dl.acm.org/doi/10.1145/1238844.1238850>

- [2] Joe Armstrong, Making reliable distributed systems in the presence of software errors, PhD thesis, KTH/SICS, 2003
https://erlang.org/download/armstrong_thesis_2003.pdf
- [3] Erlang: The Movie (Ericsson, 1990), on YouTube
<https://www.youtube.com/watch?v=xrIjfIjssLE>
- [4] Erlang/OTP official site and open-source releases (since December 1998)
<https://www.erlang.org/>
- [5] Erlang/OTP source repository
<https://github.com/erlang/otp>
- [6] Elixir, the second major BEAM language (Jose Valim, 2012-)
<https://elixir-lang.org/>

THE HISTORY OF ERLANG

This page's color band carries the
complete chapter source:
src/02-erlang.toml

Print, photograph, then decode with
PrintablePrograms.png/bin/printout.tcl:
tclsh printout.tcl decode <photo>